

Algoritmos_WilliamReneMoreno_AG2

June 30, 2026

1 03MIAR - Algoritmos de optimización

1.1 Actividad guiada 2

William Rene Moreno Romero

Colab: <https://colab.research.google.com/drive/1zSXn1t6dCvlnCz0kYZaTvB58jQOLMDz?usp=sharing>

github: https://github.com/renemorenow/VIU-AlgoritmosOptimizacion/blob/main/AG2/Algoritmos_WilliamR

1.2 Presentación

Profesor: - Grupos A2, A3, B1, B2: David Álvarez Robles (dálvarezr@professor.universidadviu.com)

1.3 Objetivos

- Comprender la utilidad y aplicar algoritmos dinámicos
- Comprender la utilidad y aplicar algoritmos de ramificación y poda
- Comprender la utilidad y aplicar el algoritmo de optimización por descenso de gradiente

1.4 Programación dinámica - Problema del viaje por el río

- Se divide el problema en subproblemas más pequeños para poder usar las soluciones más adelante
- Si el problema verifica el principio de optimalidad de Bellman “en una secuencia óptima de decisiones, toda sub-secuencia es también óptima” esto funciona bien
- Guardar soluciones parciales + recursividad = complejidad

```
[ ]: import math

i = float("inf")

tarifas = [
    [0, 5, 4, 3, i, i, i],
    [i, 0, i, 2, 3, i, 11],
    [i, i, 0, 1, i, 4, 10],
    [i, i, i, 0, 5, 6, 9],
    [i, i, i, i, 0, i, 4],
    [i, i, i, i, i, 0, 3],
```

```

    [i, i, i, i, i, i, 0]
]

# Función para obtener precios finales entre dos nodos y qué rutas seguir
def obtener_precios_y_rutas(tarifas):
    # Inicialización de tabla de precios y rutas (precios a infinito como peor
    ↪ caso)
    nodos = len(tarifas[0])
    precios = [[i]*nodos for i in [float("inf")]*nodos]
    ruta = [[i]*nodos for i in [""]*nodos]

    # Recorremos todos los caminos (desde "n" hasta un "n" superior -> Ej: 0 a
    ↪ 1, 0 a 2, 1 a 4... pero nunca de 1 a 0 o de 4 a 2)
    for i in range(nodos-1):
        for j in range(i+1,nodos):

            # Asumimos que el mínimo es la tarifa de un nodo a otro inicialmente
            minimo = tarifas[i][j]
            # Asumimos que la mejor ruta de un nodo a otro comienza en el nodo
            ↪ inicial (ej: 0,1 -> 0, 0,4 -> 0, 2,5 -> 2)
            ruta[i][j] = i

            # Recorremos posibles movimientos intermedios
            for k in range(i,j):
                #print(precios)
                #print(f"i: {i}, j: {j}, k: {k}, precios_ik: {precios[i][k]},
                ↪ tarifas_kj: {tarifas[k][j]}")
                # Si el precio de ir de i a k más la tarifa de k a j es menor
                ↪ que la tarifa de i a j, nos quedamos con esa ruta
                if (precios[i][k] + tarifas[k][j] < minimo):
                    minimo = precios[i][k] + tarifas[k][j]
                    ruta[i][j] = k
                # Actualizamos precio de i a j como el mínimo (puede ser
                ↪ pasando por k)
                precios[i][j] = minimo

    # Retornamos tabla de precios y de rutas de un nodo a otro
    return precios, ruta

# Función que calcula la ruta óptima a seguir
def calcular_ruta(ruta, inicio, fin, resultado = []):
    # Si el nodo inicial es igual al final, devolvemos el nodo
    if inicio == fin:
        #print(f"{inicio}")
        resultado.append(inicio)
        return resultado

```

```

    # Si tengo que desplazarme, calculo recursivamente la ruta de inicio a
    ↪ final siguiendo la tabla de rutas
    else:
        #print(f"{fin}")
        resultado.append(fin)
        return calcular_ruta(ruta, inicio, ruta[inicio][fin])

precios, ruta = obtener_precios_y_rutas(tarifas)

for fila in precios:
    print(fila)

print("----")

for fila in ruta:
    print(fila)

desde=0
hasta=6
resultado = calcular_ruta(ruta, desde, hasta)
print("----")
print(f"Ruta a seguir: {resultado[::-1]}")

```

```

[inf, 5, 4, 3, 8, 8, 11]
[inf, inf, inf, 2, 3, 8, 7]
[inf, inf, inf, 1, 6, 4, 7]
[inf, inf, inf, inf, 5, 6, 9]
[inf, inf, inf, inf, inf, inf, 4]
[inf, inf, inf, inf, inf, inf, 3]
[inf, inf, inf, inf, inf, inf, inf]

```

```

['', 0, 0, 0, 1, 2, 5]
['', '', 1, 1, 1, 3, 4]
['', '', '', 2, 3, 2, 5]
['', '', '', '', 3, 3, 3]
['', '', '', '', '', 4, 4]
['', '', '', '', '', '', 5]
['', '', '', '', '', '', '']

```

Ruta a seguir: [0, 2, 5, 6]

1.5 Ramificación y poda - Problema de las tareas

- Se necesitan funciones auxiliares para calcular todo el grafo (árbol)
- Función para calcular costes de soluciones parciales (no completas)
- Función para calcular cota inferior (muy optimista, repitiendo tareas si es necesario aunque la poda sea peor)
- Función para crear hijos e ir construyendo las soluciones

- Función para ramificar nodos y podar

```
[ ]: import itertools
import math

costes = [
    [11,12,18,40],
    [14,15,13,22],
    [11,17,19,23],
    [17,14,20,28]
]

#for s in itertools.permutations(range(len(costes))):
#    print(s)
```

```
[ ]: def fuerza_bruta(costes):

    optimo = float("inf")

    for s in itertools.permutations(range(len(costes))):
        coste = sum(costes[i][s[i]] for i in range(len(costes))) # Acumulo
        ↪costes de cada agente
        if (coste < optimo):
            tareas, optimo = s, coste

    return tareas, optimo

tareas, optimo = fuerza_bruta(costes)
print(f"Obtenida mejor solución {tareas} con coste {optimo} por fuerza bruta")
```

Obtenida mejor solución (0, 2, 3, 1) con coste 61 por fuerza bruta

```
[ ]: def coste_parcial(solucion, costes):
    coste = 0

    for i in range(len(solucion)):
        coste += costes[i][solucion[i]]

    return coste

#coste = coste_parcial((0,1),costes) # 11 + 15 = 26
#print(coste)
```

```
[ ]: def cota_inferior(solucion, costes):
    # Calculo el coste parcial de la solución dada
    coste = coste_parcial(solucion, costes)
```

```

    # Calculo coste mínimo de opciones restantes no usadas permitiendo
    ↪reutilización por sencillez (relajación = menos poda)
    no_usadas = set(range(len(costes)))-set(solucion)

    for i in range(len(solucion),len(costes)):
        coste += min([costes[i][j] for j in no_usadas])

    return coste

#CI = cota_inferior((0,1),costes)
#print(CI)

```

```

[ ]: # def cota_superior(solucion, costes):
#     # Calculo el coste parcial de la solución dada
#     coste = coste_parcial(solucion,costes)

#     # Calculo coste máximo de opciones restantes no usadas
#     no_usadas = set(range(len(costes)))-set(solucion)

#     for i in range(len(solucion),len(costes)):
#         coste += max([costes[i][j] for j in no_usadas])

#     return coste

# CS = cota_superior((0,1),costes)
# print(CS)

```

```

[ ]: # Función para crear las ramas del árbol
def crear_hijos(solucion, dimension):
    hijos = []
    for i in range(dimension):
        if i not in solucion:
            hijos.append({'s':solucion + (i,)})
    return hijos

#crear_hijos((0,1),4)

```

```

[ ]: def ramificacion_y_poda(costes):

    dimension = len(costes)

    # Inicializo solución a vacío con el peor coste posible
    mejor_solucion = tuple(i for i in range(dimension))
    mejor_coste = coste_parcial(mejor_solucion,costes)

    # Creo un array de nodos y lo inicializo con una solución vacía (para
    ↪comenzar por algo)

```

```

nodos = []
nodos.append({'s':(), 'ci':cota_inferior((),costes)})

# Llevo cuenta de iteraciones por debugging
iteracion = 0

# Mientras haya nodos por explorar, aplico ramificación y poda
while(nodos):
    # Cojo el nodo más prometedor (menor cota inferior, aunque esto luego
    ↪ puede ser "peor" a posteriori)
    nodo = min(nodos, key=lambda x: x['ci'])

    # Ramifico el nodo para obtener sus hijos
    hijos = crear_hijos(nodo['s'],dimension)

    # Para cada hijo, hago el procesamiento adecuado antes de decidir si es
    ↪ solución completa o un nuevo nodo
    for hijo in hijos:

        # Si la dimensión de la solución es igual a la dimensión de costes,
        ↪ es una solución final
        if (len(hijo['s']) == dimension):
            # Calculo el coste de la solución obtenida en el árbol
            coste = coste_parcial(hijo['s'],costes)
            # Si el coste es mejor, me lo quedo como solución
            if (coste < mejor_coste):
                mejor_solucion = hijo['s']
                mejor_coste = coste

            # Si la dimensión de la solución no es igual a la dimensión de
            ↪ costes, aplico poda
            else:
                # Calculo cota inferior del hijo
                cota_inferior_hijo = cota_inferior(hijo['s'], costes)
                # Si la cota inferior es menor que el mejor coste, lo añado a
                ↪ la lista de nodos a actualizar
                # Si la cota inferior fuera mayor, no tiene sentido porque no
                ↪ puede mejorar mi solución -> Poda
                if (cota_inferior_hijo <= mejor_coste):
                    nodos.append({'s':hijo['s'], 'ci':cota_inferior_hijo})
                #else:
                #print(mejor_coste)
                #print(f" Nodo eliminado: {hijo['s']} con cota inferior
                ↪ {cota_inferior_hijo}")

        # Elimino el nodo ya mirado

```

```

        nodos.remove(nodo)

        # Llevo la cuenta de las iteraciones que hago
        iteracion += 1

    return iteracion, mejor_solucion, mejor_coste

iteraciones, mejor_solucion, mejor_coste = ramificacion_y_poda(costes)
print(f"Obtenida mejor solución {mejor_solucion} con coste {mejor_coste} en {iteraciones} iteraciones")

```

Obtenida mejor solución (0, 2, 3, 1) con coste 61 en 17 iteraciones

1.6 Descenso del gradiente - Optimización de funciones

- Se lleva a cabo una optimización basada en el método del descenso de gradiente
- Se calcula el vector de derivadas parciales y se minimiza la función objetivo actualizando los nuevos puntos (ver figura)

```

[ ]: import math
import matplotlib.pyplot as plt
import numpy as np
import random

f = lambda X : X[0]**2 + X[1]**2 #  $x^2 + y^2$ 
df = lambda X : [2*X[0] , 2*X[1]] #  $(2x, 2y)$ 

resolucion = 100
rango = 2.5
X = np.linspace(-rango, rango, resolucion)
Y = np.linspace(-rango, rango, resolucion)
Z = np.zeros([resolucion, resolucion])

for ix, x in enumerate(X):
    for iy, y in enumerate(Y):
        Z[ix,iy] = f([x,y])

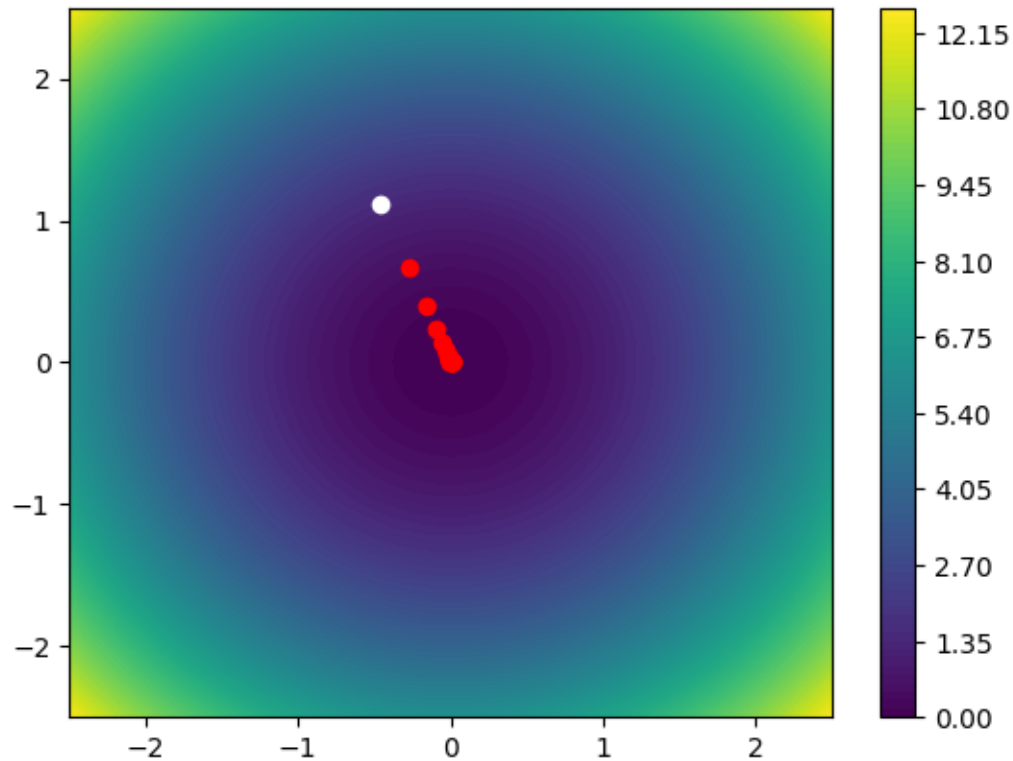
plt.contourf(X, Y, Z, resolucion)
plt.colorbar()

P = [random.uniform(-2,2), random.uniform(-2,2)]
plt.plot(P[0],P[1], "o",c="white")

tasa_aprendizaje = 0.2
for _ in range(500):
    grad = df(P)
    P[0], P[1] = P[0] - tasa_aprendizaje*grad[0], P[1] -
    tasa_aprendizaje*grad[1]

```

```
plt.plot(P[0],P[1],"o",c="red")
```



1.7 Actividades extra

- Ramificación y poda
 - Generar matrices con valores aleatorios de mayores dimensiones (5,6,7,...) y ejecutar ambos algoritmos.
 - ¿A partir de que dimensión el algoritmo por fuerza bruta deja de ser una opción?
 - ¿Hay algún valor de la dimensión a partir de la cual el algoritmo de ramificación y poda también deja de ser una opción válida?
- Descenso de gradiente
 - Minimizar la función $\sin(1/2 * x^2 - 1/4 * y^2 + 3) * \cos(2*x + 1 - E^y)$

1.7.1 - Ramificación y poda

- Generar matrices con valores aleatorios de mayores dimensiones (5,6,7,...) y ejecutar ambos algoritmos.

```
[ ]: import time
```

```
[ ]: def generar_matriz_ndim(val_dimension):  
    return np.random.randint(10, 100, size=(val_dimension, val_dimension))
```



```
[ ]: # Generación de matrices desde 5 hasta 10 dimensiones:
for j in range(5, 11):
    matriz_j = generar_matriz_ndim(j)
    print(matriz_j)

    # 1. Ejecución con fuerza_bruta:
    t0 = time.time()
    coste_fb = fuerza_bruta(matriz_j)
    tiempo_fb = time.time() - t0
    str_fb = f"{tiempo_fb:.5f}"

    # 2. Ejecución con ramificacion_y_poda:
    t0 = time.time()
    coste_rp = ramificacion_y_poda(matriz_j)
    tiempo_rp = time.time() - t0
    mismo_resultado = (coste_fb == coste_rp) if coste_fb is not None else "N/A"
    print(f"Dimensión: {j} | Tiempo Fuerza Bruta: {str_fb:9s} | Tiempo_
↪Ramificación y Poda: {tiempo_rp:9.5f} | Resultado igual?:_
↪{mismo_resultado}\n")
```

```
[[96 52 78 61 66]
 [38 38 97 26 85]
 [96 81 36 97 30]
 [64 85 69 21 60]
 [56 23 66 79 27]]
```

Dimensión: 5 | Tiempo Fuerza Bruta: 0.00157 | Tiempo Ramificación y Poda:
0.00085 | Resultado igual?: False

```
[[91 96 89 39 56 82]
 [67 54 69 72 58 99]
 [98 50 10 11 27 80]
 [22 12 30 59 95 38]
 [46 28 80 51 40 68]
 [76 29 55 38 22 71]]
```

Dimensión: 6 | Tiempo Fuerza Bruta: 0.00438 | Tiempo Ramificación y Poda:
0.00295 | Resultado igual?: False

```
[[10 73 66 41 55 99 94]
 [67 60 21 99 77 53 68]
 [98 57 19 35 24 18 82]
 [55 69 95 17 92 42 85]
 [38 98 85 48 68 64 46]
 [69 88 37 76 35 89 68]
 [24 45 82 61 71 45 60]]
```

Dimensión: 7 | Tiempo Fuerza Bruta: 0.03393 | Tiempo Ramificación y Poda:
0.00277 | Resultado igual?: False

```
[[52 48 41 49 53 34 82 36]
 [40 47 32 28 45 28 96 27]
 [88 18 11 42 78 76 73 71]
 [49 32 74 12 40 71 89 29]
 [10 41 53 38 76 61 17 93]
 [55 89 36 70 16 77 63 19]
 [54 79 11 21 77 27 37 79]
 [56 69 90 96 74 24 72 14]]
```

Dimensión: 8 | Tiempo Fuerza Bruta: 0.35769 | Tiempo Ramificación y Poda:
0.02199 | Resultado igual?: False

```
[[83 15 44 16 93 33 41 56 70]
 [26 42 14 21 44 76 81 29 28]
 [50 54 49 76 41 74 41 49 82]
 [98 50 85 48 49 27 73 23 12]
 [42 52 79 61 25 32 53 23 46]
 [61 59 86 69 76 69 50 84 74]
 [59 14 88 32 52 57 38 48 84]
 [17 90 24 45 98 86 58 51 43]
 [35 81 55 10 67 99 23 44 70]]
```

Dimensión: 9 | Tiempo Fuerza Bruta: 1.44618 | Tiempo Ramificación y Poda:
0.00945 | Resultado igual?: False

```
[[50 80 17 24 27 23 52 85 79 23]
 [44 70 14 41 90 98 61 19 49 30]
 [57 83 28 34 27 23 73 27 14 83]
 [70 87 64 92 25 58 50 31 32 51]
 [74 31 64 53 26 50 25 62 44 26]
 [57 40 12 62 65 19 48 99 25 22]
 [27 61 13 57 69 11 40 88 46 62]
 [65 58 26 57 65 22 31 22 47 83]
 [49 92 23 61 24 93 83 19 19 96]
 [62 56 28 19 68 22 21 40 39 82]]
```

Dimensión: 10 | Tiempo Fuerza Bruta: 18.45898 | Tiempo Ramificación y Poda:
0.23335 | Resultado igual?: False

- ¿A partir de que dimensión el algoritmo por fuerza bruta deja de ser una opción?

De acuerdo a las pruebas realizadas se concluye para este caso que a partir de la dimensión 10 u 11, el tiempo de computo en fuerza bruta se escala exponencialmente, debido al crecimiento factorial (Ej: $11! = 39,916,800$ operaciones de asignación)

- ¿Hay algún valor de la dimensión a partir de la cual el algoritmo de ramificación y poda también deja de ser una opción válida?

Este algoritmo se comporta de manera excelente con las dimensiones dadas, pero cuando la matriz tiene valores muy homogéneos y las cotas estimadas no logran podar ramas eficazmente, se degrada a una exploración completa de estados. En los experimentos realizados a partir de dimensiones entre 14 o 15, el consumo de

memoria o el tiempo de ejecución se saturan, lo cual también deja de ser una opción válida para dar paso a otros como metaheurísticos.

1.7.2 - Descenso de gradiente

- Minimizar la función $\sin(1/2 * x^2 - 1/4 * y^2 + 3) * \cos(2x + 1 - e^y)$

Utilizando la regla del producto y la regla de la cadena:

Derivada respecto a x:

$$\frac{\partial f}{\partial x} = x \cdot \cos\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \cdot \cos(2x + 1 - e^y) - 2 \cdot \sin\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \cdot \sin(2x + 1 - e^y)$$

Derivada respecto a y:

$$\frac{\partial f}{\partial y} = -\frac{1}{2}y \cdot \cos\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \cdot \cos(2x + 1 - e^y) + e^y \cdot \sin\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \cdot \sin(2x + 1 - e^y)$$

El algoritmo iniciará en un punto aleatorio (x_0, y_0) . Para actualizar sus coordenadas de forma iterativa, resta el gradiente multiplicando por una tasa de aprendizaje (α)

```
[ ]: def f_2(P):
    x, y = P[0], P[1]
    return np.sin(0.5*x**2 - 0.25*y**2 + 3) * np.cos(2*x + 1 - np.exp(y))

def df_2(P):
    x, y = P[0], P[1]
    arg_sin = 0.5*x**2 - 0.25*y**2 + 3
    arg_cos = 2*x + 1 - np.exp(y)

    # Generar nuevas derivadas parciales analíticas:
    grad_x = (x * np.cos(arg_sin) * np.cos(arg_cos) - 2*np.sin(arg_sin) * np.
    ↪sin(arg_cos))
    grad_y = (-0.5*y * np.cos(arg_sin) * np.cos(arg_cos) + np.exp(y) * np.
    ↪sin(arg_sin) * np.sin(arg_cos))

    return np.array([grad_x, grad_y])
```

```
[ ]: X_vals = np.linspace(-2, 2, 200)
Y_vals = np.linspace(-2, 2, 200)
X_mesh, Y_mesh = np.meshgrid(X_vals, Y_vals)

# Evaluación de la nueva función sobre la cuadrícula:
Z = f_2([X_mesh, Y_mesh])

# Dibujar el mapa de contornos (Curvas de nivel):
```

```

plt.figure(figsize=(10, 8))
plt.contourf(X_mesh, Y_mesh, Z, 50, cmap='viridis')
plt.colorbar(label='Valor de f_2(x,y)')

# Inicialización aleatoria del punto P dentro del plano:
P = [random.uniform(-2, 2), random.uniform(-2, 2)]
plt.plot(P[0], P[1], "o", c="white", label="Inicio", markersize=10)

# Mantenemos los parámetros estándar del ejercicio:
tasa_aprendizaje = 0.05
iteraciones = 500

for _ in range(iteraciones):
    grad = df_2(P) # Evaluación del nuevo gradiente analítico
    P[0] = P[0] - tasa_aprendizaje * grad[0]
    P[1] = P[1] - tasa_aprendizaje * grad[1]

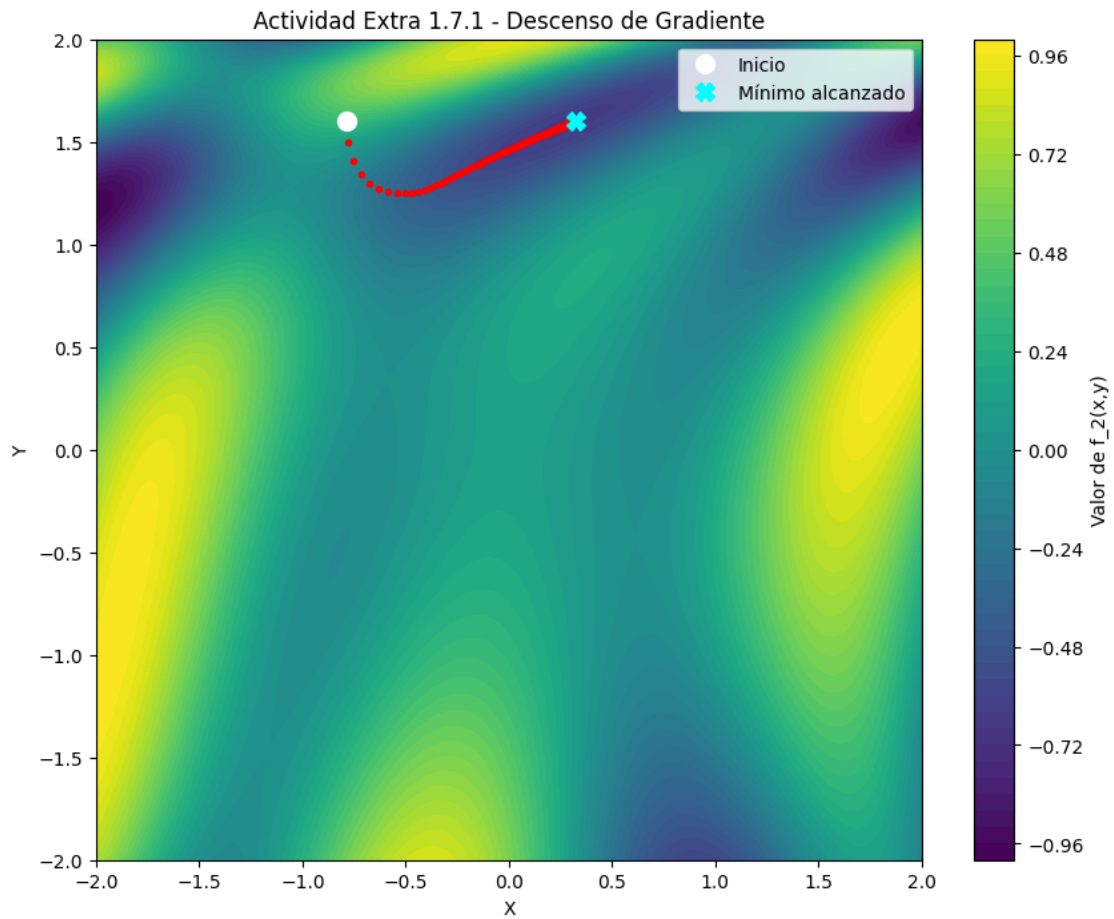
    # Dibujo del camino (rojo):
    plt.plot(P[0], P[1], "o", c="red", markersize=3)

# Para marcar el mínimo local alcanzado:
plt.plot(P[0], P[1], "X", c="cyan", label="Mínimo alcanzado", markersize=10)

plt.title("Actividad Extra 1.7.1 - Descenso de Gradiente")
plt.xlabel("X")
plt.ylabel("Y")
plt.xlim(-2, 2)
plt.ylim(-2, 2)
plt.legend()
plt.show()

print(f"Coordenadas del mínimo local encontrado: X = {P[0]:.4f}, Y = {P[1]:.4f}")
print(f"Valor mínimo de la función: {f_2(P):.4f}")

```



Coordenadas del mínimo local encontrado: $X = 0.3227$, $Y = 1.6024$
Valor mínimo de la función: -0.6574